

Nome: Federico
Cognome: Triglia

RELAZIONE PROGETTO JBLACKJACK

SPECIFICHE DI PROGETTAZIONE

In seguito sarà illustrata l'implementazione delle specifiche del progetto, tenendo conto di tutti i punti della consegna.

GESTIONE DEI PROFILI UTENTE

La gestione dei profili utente si basa fondamentalmente su 3 classi e una enumerazione: "Profile", "Database", "Menu", e "Avatar". Il funzionamento di tali classi viene illustrato di seguito. Tuttavia, in questa parte saranno trattati gli aspetti di progettazione relativi queste classi esclusivamente nel contesto del modello. Gli aspetti relativi la view e il controller saranno trattati più avanti.

LA CLASSE "Profile"

Per capire bene la gestione dei profili utente è bene partire dal comprendere la classe "Profile": tramite questa classe, un utente può istanziare un proprio profilo così da poter giocare. Tale classe è costituita dai seguenti campi:

- Esattamente un "nickname" (stringa), che il software realizzato garantisce sia univoco (tale campo viene usato chiave primaria).
- Senza passare questo valore al costruttore, infatti, la classe non è istanziabile.
- Inoltre tale campo non è provvisto, per ovvie ragioni, di un relativo setter.
- Esattamente un "avatar" (enumeration Avatar), che rappresenta l'utente.
- Anche questo parametro fa parte del costruttore, e ogni profilo deve sempre averne uno.
- L'avatar è modificabile col metodo "setAvatar".
- I campi "handsWon", "handsLost", "handsPush", "handsBJ", interi che indicano rispettivamente il numero di mani vinte/perse/pareggiate/vinte con black jack.
- Ovviamente tali campi non sono compito del costruttore, in quanto al momento della creazione del profilo si sfruttano i meccanismi di inizializzazione implicita.
- Il campo "score", che indica il punteggio del giocatore, e che può essere utilizzato per effettuare puntate durante le partite.
- Tale campo serve anche a rappresentare il livello del giocatore (si presuppone che più un giocatore è bravo, più sarà alto questo valore).
- Inizialmente viene impostato a 100.
- serialVersionUID, in quanto la classe implementa l'interfaccia "Serializable".

Ad eccezione del campo `serialVersionUID`, la classe offre getter relativi ogni campo, e setter relativo l'avatar, così da poterlo cambiare. Inoltre presenta metodi che consentono di aggiornare, insieme al punteggio del giocatore, il numero di partite vinte/perse/pareggiate/vinte con blackjack (es. `void won(int bet)` aumenta di 1 il valore del campo `handsWon`, e aggiorna il campo `score` col valore del parametro `bet`). Il metodo `toString` torna una rappresentazione a stringa dell'oggetto, considerando tutti i campi non statici che lo compongono. Inoltre è presente il metodo `castFromString` (statico, che torna un profilo). Il funzionamento di tale metodo sarà illustrato più avanti insieme alla spiegazione della classe `Database`.

L'ENUMERAZIONE "Avatar"

Abbiamo visto come la classe `Profile` faccia uso della enum `Avatar`. Per cui, per averne una panoramica completa, è bene trattare anche questo argomento. Le costanti enumerative utilizzate sono esattamente una relativa il dealer del gioco (`Avatar.DEALER`), e svariate relative agli animali (es. `Avatar.DOG`, `Avatar.CAT`, `Avatar.RABBIT`), intese per essere scelte dagli utenti. Come da prassi è stato effettuato un override di `toString`, che torna il nome della costante enumerativa in Pascal-case. La enum inoltre presenta il metodo statico `fromString` che, data una stringa, se tale stringa, a prescindere dal case dei caratteri, coincide con l'output del metodo `name` di una delle costanti, allora ritorna la enum in questione. Tale metodo è stato realizzato facendo uso degli stream. Altro metodo è `getAnimals`, anche questo realizzato con gli stream, che torna un array contenente l'output di `name` di ogni costante enumerativa ad eccezione di `Avatar.DEALER`.

LA CLASSE "Menu"

La classe `Menu` offre all'utente varie funzionalità relative le impostazioni di gioco, ma le principali riguardano la gestione dei profili. Tale classe presenta il campo `currentUser`, che potrebbe essere null se non esiste nessun profilo utente (ad esempio, quando il software viene avviato per la prima volta). È presente anche il campo `dealerProfile`, relativo il profilo del dealer, che ha un nickname e un avatar riservati. Inoltre il campo `userProfiles` (`List<Profile>`), contiene tutti i profili utente creati, e tra i questi, in fase iniziale, l'utente può sceglierne uno in modo da poter avviare la partita. Altri campi relativi le impostazioni di gioco sono il campo `decksNumber` (un intero tra 2 e 6 indicante il numero di mazzi di una eventuale partita), e `americanRules` (un booleano indicante se si vuole giocare al blackjack americano o europeo). Come detto, la classe consente la gestione dei profili utente, e infatti ci sono molti metodi che se ne occupano, come `hasProfile` (se quel profilo esiste), `addProfile` (aggiunge un profilo al campo `userProfiles`), `setCurrentUser` (per cambiare l'utente scelto per una eventuale partita), `getCurrentUser`, `hasProfile` (dato il nickname, torna il profilo relativo, se esiste), `setAvatar` (per impostare l'avatar dell'utente corrente, se esiste), `getDealerProfile`, `getUsers`, `deleteProfile` (rimuove da `userProfiles` il profilo in questione), `setProfiles` (per impostare il campo `userProfiles`), e `getNicks`, realizzato con gli stream, che torna un array di stringhe relative i nickname dei profili utente. Altri metodi sono getter e setter relativi numero di mazzi e regole di gioco. Inoltre sono presenti diversi metodi, oltre quelli già illustrati, che fanno uso degli stream, ma tale aspetto sarà ripreso in seguito. La caratteristica più importante di questa classe, ai fini del funzionamento del gioco, è l'estensione della classe `Observable`. Gli aspetti relativi tale pattern e l'uso dell'architettura MVC saranno ripresi in seguito, tuttavia è bene sottolineare come questa classe rappresenti l'intera logica relativa la prima fase di avvio del software, offrendo all'utente la possibilità di decidere in che modo giocare. Proprio per questo, è stato deciso di adottare il pattern singleton su questa classe.

LA CLASSE "Database"

La classe "Database" serve per effettuare il salvataggio/caricamento dei dati su/da disco. Per congruenza, anche questa classe è stata resa singleton. I metodi principali di questa classe sono "save" e "load". "save" crea, se non presenti, la directory "data" e il file "JBJ.data" (il nome di tale file è un campo static e final di tipo String) al suo interno, e, facendo uso del metodo "toString" della classe "Profile", vi salva i profili presenti nel campo "profiles" (List<Profile>). Quest'ultimo campo può essere aggiornato mediante i metodi "addProfile" e "removeProfile". "load" si occupa invece del caricamento dei dati da disco, e fa uso del metodo statico "castFromString" della classe "Profile". Quest'ultimo metodo, data una stringa, se tale stringa contiene una rappresentazione di un profilo, allora crea e torna il profilo in questione (effettua quindi l'operazione opposta a "toString"). Altro metodo di questa classe è "getProfiles", che torna il campo "profiles". Anche se i dettagli relativi all'implementazione di MVC saranno illustrati in seguito, si sottolinea come la classe Database non interagisce mai direttamente con le classi del modello: è il controller a fare effettuare alla classe il caricamento dei dati da disco e poi passarli al modello (caricamento), o prendere i dati dal modello e passarli alla classe per il salvataggio (salvataggio).

GESTIONE DI UNA PARTITA COMPLETA

In seguito si tratteranno gli aspetti del modello relativi alla gestione di una partita completa. Essendovi diverse classi adibite a questo compito, per maggiore chiarezza si è preferito scomporre tale trattazione in più parti: la prima relativa alla gestione delle carte da gioco, la seconda relativa alla gestione dei giocatori, e l'ultima relativa alla logica di gioco complessiva.

GESTIONE DELLE CARTE

Vi sono 2 classi e 2 enumerazioni adibite a tale gestione: "Card", "Deck", "Suit", "Rank". Prima di illustrare ogni singola classe, si fa notare come è stato scelto di rappresentare una relazione di composizione tra la classe "Card" da un lato, e le 2 enum "Suit" e "Rank" dall'altro. Nel mini-mondo del blackjack, infatti, una carta ha senso di esistere solo se vi si associano una costante che ne indichi il seme e una che ne indichi il valore. È stata inoltre introdotta una relazione di composizione anche tra la classe "Deck" e "Card", per gli stessi motivi.

La enum "Suit"

Una enum semplice, che rappresenta i quattro semi possibili e relativa rappresentazione a stringa in Pascal-case (col metodo toString).

La enum "Rank"

Questa enumeration presenta una costante per ogni possibile valore e/o figura che una carta può assumere. Come da regolamento, ogni figura vale 10, mentre a ogni altra costante è associato il relativo valore numerico. L'unica eccezione è costituita dall'asso. Di default è stato assegnato a questa costante il valore 11, tuttavia a seconda della situazione può valere anche 1 (questa funzionalità non è stata considerata responsabilità di questa enum, in quanto il cambio di valore dipende dal contesto di gioco). Sono presenti il campo value (un intero), il metodo "toInt()", che torna il valore associato, e l'override di "toString".

La classe "Card"

Tale classe rappresenta una carta da gioco. È costituita da 3 campi che indicano seme (Suit), valore (Rank), e se la carta è coperta o meno (boolean). Essendo i primi 2 campi final sono presenti solo i relativi getter. È stato effettuato poi l'override di "toString" come da prassi, e sono

stati inseriti appositi metodi per coprire/scoprire la carta o per sapere se è coperta/scoperta. Sono stati inoltre effettuati gli override di "equals" e "hashCode", basandosi su seme e valore della carta. Inoltre tale classe implementa l'interfaccia "Comparable", basandosi sul valore della carta per effettuare il confronto.

La classe "Deck"

Questa classe rappresenta un mazzo di carte (per essere più precisi, un insieme di mazzi che vengono uniti per formarne uno solo). Presenta infatti un campo "deck" (`List<Card>`), un metodo "shuffle", che mescola il mazzo, e un metodo "drawCard", che estrae una carta dal mazzo. Inoltre il costruttore prende in input un intero fra 2 e 6 che indica il numero di mazzi effettivi che costituiscono l'istanza di questa classe (infatti, secondo il regolamento del gioco, è necessario giocare con almeno 2 mazzi e non più di 6). Tale classe è inoltre iterabile (implementa l'interfaccia "Iterable<Card>").

GESTIONE DEI GIOCATORI

Per quanto riguarda la gestione dei giocatori, è stato fatto utilizzo delle classi "Player", "User", "Dealer", "Hand", e dell'interfaccia funzionale "GameStrategy".

La classe "Player"

Questa classe astratta rappresenta un generico giocatore. È bene sottolineare la differenza con la classe "Profile": un'istanza di "Player" presenta un campo "profile", final da passare al costruttore, che rappresenta il relativo profilo. Tuttavia le 2 classi non svolgono la stessa funzione: mentre un profilo ha lo scopo di mantenere i dati di un utente, e infatti è pensato per essere usato nella fase iniziale di gioco, un'istanza di "Player" invece è pensata per essere usata durante la partita vera e propria. Proprio per questo, contiene anche un campo "hand" (la mano attuale del giocatore), e un campo "strategy" (la strategia attuale del giocatore). I metodi di questa classe sono:

- getHand: torna la mano del giocatore
- getAvatar: torna l'avatar del profilo del giocatore
- getStrategy: torna la strategia del giocatore
- play: esegue l'azione della strategia del giocatore
- reset: prepara il giocatore a un nuovo turno
- isBusted: indica se il giocatore ha "sballato" (valore della mano ≥ 21)
- handValue: torna il valore della mano del giocatore. Si basa sull'omonimo metodo della classe "Hand"
- addCard: aggiunge una carta alla mano del giocatore
- getProfile: torna il profilo del giocatore
- toString (override)

La classe "Hand"

Questa classe rappresenta una mano di un giocatore. Infatti, è costituita dal campo "cards" (List<Card>), che rappresenta le carte della mano. Inoltre implementa l'interfaccia Comparable<Hand>: il confronto si basa principalmente sul metodo "handValue()", che torna il valore della mano. È stata implementata anche l'interfaccia Iterable<Card>, per rendere la classe iterabile. Inoltre la classe presenta i metodi "size", "clear", "get", e "remove", che effettuano l'omonima operazione sul campo dell'istanza. È presente anche il metodo "splittable", che torna un booleano e indica se la mano può essere splittata, e il metodo "addCard", che aggiunge una carta alla mano.

La classe "User"

Questa classe estende "Player" e rappresenta l'utente. È stata resa singleton. Il motivo di tale scelta è dovuto al fatto che nel blackjack, a differenza di molti altri giochi di carte, anche se ci sono più persone a giocare contemporaneamente è come se ognuno giocasse singolarmente contro il dealer. Per questo, quindi, è stato ritenuto superfluo consentire a più giocatori umani di partecipare alla stessa partita. I campi di cui fa utilizzo sono:

- bet (indica la puntata attuale del giocatore)
- secondBet (indica la puntata attuale del giocatore sulla seconda mano, nel caso in cui il giocatore abbia effettuato uno split)
- secondHand (la seconda mano del giocatore, nel caso in cui questo abbia splittato, altrimenti tale campo ha valore "null")
- currentSecond (un booleano che indica la mano corrente del giocatore)

I metodi della classe non presenti nella superclasse sono:

- push/BJ/won/lost (aggiornano il profilo del giocatore)
- getBet (torna la puntata del giocatore)
- split/doubleDown (il giocatore effettua split/double down)
- currentHand (imposta la mano attuale del giocatore)
- getSecondHand (torna la seconda mano del giocatore)
- setBet (imposta la puntata iniziale del giocatore)
- setSecondBet (imposta la puntata della seconda mano)
- getSecondBet (torna la seconda puntata del giocatore)
- getCurrentBet (torna la puntata della mano corrente)
- setStrategy (imposta la strategia del giocatore. Nota: il setter è stato messo solo in "User" nonostante il campo appartenga a "Player", perché la strategia del dealer è predefinita e quindi non modificabile)
- lastHand (true se l'utente sta giocando l'ultima mano del suo turno)
- playingSecond (true se l'utente sta giocando la seconda mano)
- splittable/doubleDownable (true se l'utente può fare split/double down)

La classe "Dealer"

Questa classe rappresenta il croupier/dealer del gioco, ed estende la classe "Player". È stata resa singleton per congruenza con le regole di gioco, e presenta un campo relativo al mazzo di gioco, e un campo relativo alle regole di gioco che il dealer deve garantire. I metodi del dealer non presenti in "Player" sono:

- shuffle: per mischiare il mazzo (chiama il metodo omonimo della classe "Deck")
- giveCard: per dare una carta del mazzo a un giocatore
- discoverSecond: per scoprire la seconda carta nel caso sia coperta
- canPlay: true se il dealer può giocare contro l'utente passato come argomento del metodo
- toPlayAgainst: metodo privato per semplificare il metodo canPlay

L'interfaccia "GameStrategy"

Questa interfaccia funzionale è cruciale per l'adozione del pattern strategy, che è stato usato per rappresentare varie possibili strategie adottabili durante il gioco. In particolare, è estesa dall'interfaccia UserStrategy, la quale distingue le strategie possibili dell'utente ed è implementata da "HitStrategy", "DoubleDownStrategy", "SplitStrategy", e "StandStrategy". L'interfaccia "GameStrategy" è anche implementata da "DealerStrategy", che rappresenta la strategia del dealer. Il metodo astratto di questa interfaccia è "play", che torna un booleano nel caso la strategia di gioco sia stata eseguita senza problemi. Le classi che implementano questa interfaccia sono:

- DealerStrategy: definisce il comportamento relativo alla strategia del dealer
- HitStrategy: definisce il comportamento relativo alla strategia di "hit" dell'utente
- StandStrategy: definisce il comportamento relativo alla strategia di "stand" dell'utente
- SplitStrategy: definisce il comportamento relativo alla strategia di "split" dell'utente
- DoubleDownStrategy: definisce il comportamento relativo alla strategia di "double down" (raddoppio) dell'utente

Tutte le strategie tranne quella del dealer implementano l'interfaccia tramite "UserStrategy", che svolge la funzione di marcatore.

LOGICA DI GIOCO

In seguito saranno illustrate la enum "Result" e la classe "Game", usata per rappresentare la logica di gioco.

La enum "Result"

Questa enum rappresenta il risultato di una partita di blackjack. Le costanti sono quindi Result.YOU_WON (se l'utente ha vinto), Result.PUSH (se l'utente ha pareggiato), Result.YOU_LOST (se l'utente ha perso), e Result.BJ (se l'utente ha vinto con blackjack, cioè ha realizzato 21 esclusivamente con le prime 2 carte che gli sono state date, che quindi sono per forza un asso e una carta di valore 10).

La classe "Game"

Tale classe rappresenta la logica di gioco complessiva. Anche se l'uso di MVC+Observer sarà trattato in seguito, anche questa classe è osservabile come la classe "Menu". Questo perché "Menu" riguarda la fase iniziale del gioco, mentre questa classe rappresenta la logica di una partita vera e propria. È stata realizzata facendo uso del pattern singleton. I campi della classe sono:

- result: il risultato della partita giocata (null se ancora non determinato)
- americanRules: true se le regole di gioco sono americane, false se europee
- dealer: il dealer
- user: l'utente che sta giocando

I metodi della classe sono:

- getRules: getter del campo "americanRules"
- getResult: getter del campo "result"
- shuffle: fa mescolare le carte al dealer
- giveCards: fa distribuire le carte al dealer
- splittable/doubleDownable: true se l'utente può effettuare uno split/double down
- getDealer/getUser
- discoverSecond: il dealer scopre la seconda carta
- getUserBet: chiama il metodo "getBet" dell'utente
- setResultHand: torna il risultato di una mano dell'utente
- setResult: metodo privato ausiliario a setResultHand
- removeResult: annulla il risultato
- reset: azzera le statistiche della partita per poterne ricominciare una nuova
- initialized: metodo statico che indica se la singola istanza della classe è stata inizializzata

DESIGN PATTERN ADOTTATI

Di seguito saranno trattati i vari design pattern usati nella realizzazione dell'applicazione, illustrando dove sono stati usati e per quale motivo si è scelto di usarli.

MVC CON OBSERVER-OBSERVABLE

Il progetto è stato suddiviso in 3 package: model, view e controller.

Il controller Il controller contiene le classi che implementano l'interfaccia ActionListener di Swing. All'interno del metodo Main.main queste classi vengono opportunamente istanziate e collegate ai relativi componenti della view. Inoltre, il controller carica eventuali dati presenti su disco, crea il modello, crea la view, li mette in comunicazione sfruttando il pattern Observer-Observable, crea gli ActionListener e li aggancia alla view (tutto ciò avviene all'interno del metodo main, che contiene anche il game loop).

Il model Il model contiene la classe "Database", adibita al salvataggio/caricamento dei dati, e la logica, illustrata nella trattazione della gestione dei profili utente e della gestione di una partita completa. Inoltre presenta 3 classi che estendono la classe astratta Observable: "Menu", relativa la prima fase di gioco, e "Game" e "Player", che insieme vengono sfruttate dalla view per aggiornarsi.

La view La view, realizzata con Swing, si occupa degli aspetti interattivi dell'applicazione (essenzialmente grafica e audio). Per quanto concerne l'audio, si avvale della classe AudioManager (singleton). Per quanto concerne la grafica, è stato tutto centralizzato sfruttando la classe "MainView" (unica classe della view che implementa l'interfaccia Observer), che aggiorna opportunamente l'intera view. La classe "MainView" può avere 2 tipi di contentPane: MenuPanel e GamePanel. Tramite il metodo MainView.setGamePanel, chiamato dal controller, viene effettuato il cambio di contentPane da MenuPanel a GamePanel. Dopo il cambiamento, il controller si occupa di eliminare il meccanismo Observer-Observable tra la classe model.Menu e la classe view.MainView. In questo modo viene effettuato il passaggio al gioco vero e proprio.

DECORATOR PATTERN

È stato fatto uso del decorator pattern all'interno della view per la visualizzazione del risultato. In particolare, è stata realizzata la classe astratta "ResultPanel", che estende JPanel, ed è a sua volta estesa da "ResultBasePanel", che definisce dei comportamenti base per la visualizzazione del JLabel che rappresenta il risultato. Inoltre, la classe "ResultPanelDecorator" estende "ResultPanel", e definisce il contratto relativo la decorazione di un "ResultPanel", basandosi su un'istanza di "ResultPanel" che deve essere presa in input dal costruttore. Infatti, la classe "ResultPanelDecorator" è estesa da 3 classi, che rappresentano 3 tipi diversi di decorazioni concrete della classe "ResultPanel":

- ResultColorDecorator: colora il testo del risultato in maniera differente a seconda del contenuto del JLabel dell'oggetto "parassitato"
- ResultTypingDecorator: mostra il risultato, ma un carattere alla volta, dando così l'effetto "typing"
- ResultFadeDecorator: sfuma l'opacità del colore del JLabel, così da farlo sparire gradualmente

Questo pattern viene sfruttato dalla classe GamePanel, che infatti nel costruttore crea il "resultPanel" in modo tale che tutti i comportamenti di decorazione vengano utilizzati (nell'ordine typing -> colorazione -> sfumatura).

STRATEGY PATTERN

È stato fatto uso dello strategy pattern all'interno del modello, tramite l'interfaccia funzionale "GameStrategy", le classi/interfacce che la implementano/estendono, e la classe astratta "Player", che infatti "has-a" "GameStrategy" (campo strategy). Tra i vari vantaggi dell'adozione di questo pattern vi è, per esempio, il non aver dovuto creare metodi astratti in Player relativi "hit" e "stand", e poi dover scrivere 2 differenti implementazioni per User e Dealer. In questo modo si raggiunge un compromesso fra manutenibilità del codice e riuso. Infatti, una volta impostata la strategia del giocatore, basterà fare uso del metodo play per effettuare l'azione prevista dal campo "strategy".

SINGLETON PATTERN

È stato fatto uso diverse volte del pattern singleton perché lo si riteneva consono con la natura della classe in questione. In particolare, è stato adottato sulle classi: `model.Menu`, `view.MainView`, `view.AudioManager`, `model.Game`, `model.User`, `model.Dealer`, `model.Database`.

USO DI JAVA SWING PER LA GUI

Java Swing è il framework utilizzato da questa applicazione per la gestione della grafica. Di seguito saranno trattate le varie classi utilizzate. Per maggiore chiarezza, la trattazione è stata scomposta in 2 parti principali: la prima relativa la grafica del menu di gioco, e la seconda relativa la grafica della partita.

GRAFICA DEL MENU

Il menu è rappresentato dalla classe "MenuPanel" (JPanel), che contiene 3 pannelli principali, disposti tramite un BorderLayout:

- StartingPanel (BorderLayout.NORTH): il pannello contenente il JButton per l'avvio di una partita. Il JButton di questo pannello viene collegato dal controller a un NewGameActionListener, che cambia il contentPanel della "MainView" e dà inizio alla partita.
- SettingsPanel (BorderLayout.WEST): il pannello relativo le impostazioni di gioco. Questo pannello contiene vari componenti disposti tramite l'utilizzo di un GridBagLayout. Inoltre, quando opportuno, il controller mette in comunicazione i componenti di questo pannello con i relativi ActionListener.
- ProfilesPanel (BorderLayout.CENTER), che fa uso della classe JTable e ProfileTableModel (che estende AbstractTableModel). In questo modo, questo pannello mostra una tabella dei vari utenti insieme alle relative statistiche.

GRAFICA DELLA PARTITA

La partita è rappresentata dalla classe "GamePanel" (JPanel), che contiene 3 campi:

- resultPanel (ResultPanel), che viene creato dal costruttore di questa classe sfruttando il pattern Decorator, già illustrato precedentemente.
- dealerPanel (DealerPanel), che estende PlayerPanel (abstract).
- userPanel (UserPanel), che estende PlayerPanel (abstract), e vi aggiunge un ButtonPanel.

Per comprendere più nel dettaglio DealerPanel e UserPanel saranno trattate nello specifico le classi PlayerPanel e ButtonPanel. ButtonPanel è un pannello pensato per essere usato dal giocatore (User). Si trova infatti come campo della classe UserPanel (BorderLayout.SOUTH), e contiene i bottoni per l'avvio di una nuova partita, il settaggio della puntata, e le azioni di gioco HIT, STAND, SPLIT, e DOUBLEDOWN. I vari componenti di questo pannello sono stati disposti facendo uso di un FlowLayout. La classe PlayerPanel rappresenta un pannello di un giocatore, e contiene come campi un'istanza di AvatarPanel e un'istanza di CardPanel. Imposta inoltre il Layout a BorderLayout, ma non dispone i componenti: ciò sarà compito delle sue implementazioni concrete UserPanel e DealerPanel. AvatarPanel è un pannello rappresentante l'immagine avatar del giocatore, e viene disposto a BorderLayout.NORTH dentro DealerPanel, e a BorderLayout.CENTER dentro UserPanel. Questo pannello carica l'immagine del giocatore come JLabel, scegliendo tra le risorse di gioco l'immagine appropriata in base alla costante Avatar passata in input al costruttore. La classe CardPanel è un pannello (JPanel) che ha lo scopo di mostrare le carte di un giocatore (rappresentate come JLabel) tramite un FlowLayout.

È disposto a BorderLayout.NORTH dalla classe UserPanel, e a BorderLayout.CENTER dalla classe DealerPanel. ResultPanel è il pannello relativo la visualizzazione del risultato. Questo pannello si trova al centro del layout del GamePanel, e viene istanziato all'interno di quest'ultima facendo uso del pattern Decorator.

UTILIZZO DEGLI STREAM

Di seguito i metodi che fanno utilizzo degli stream:

- model.Avatar.getAnimals (operazioni usate: filter, map, toArray)
- model.Avatar.fromString (operazioni usate: filter, findFirst, orElse)
- model.Menu.getNicks (operazioni usate: filter, map, toArray)
- model.Menu.getMapPlayersScore (operazioni usate: collect, Collectors.toMap)
- model.Menu.getMapAvatarSetPlayers (operazioni usate: collect, groupingBy, toSet)
- model.Menu.getSetPlayersForAvatar (operazioni usate: filter, collect, toSet)
- model.Menu.getMapScoreSetPlayers (operazioni usate: collect, groupingBy, toSet)
- model.Menu.getTotalRanking (operazioni usate: sorted, map, collect, Collectors.toList)
- model.Menu.getWonRanking (operazioni usate: sorted, map, collect, Collectors.toCollection)
- model.Menu.getBJRanking (operazioni usate: sorted, map, collect, Collectors.toList)
- model.Menu.getPlayedRanking (operazioni usate: sorted, map, collect, Collectors.toList)
- model.Menu.getMinTotalRanking (operazioni usate: filter, sorted, map, collect, Collectors.toList)

Dei metodi illustrati, tutti tranne i primi 3 servono a riportare statistiche basate sui profili del menu, e sono stati inseriti esclusivamente per motivi didattici. I primi 3 metodi, invece, sono proprio usati a runtime dall'applicazione. In particolare, "model.Avatar.fromString" viene usato dal metodo statico "model.Profile.castFromString", che è a sua volta chiamato dalla classe database nel caso il file contenente i dati presenta informazioni da leggere. "model.Avatar.getAnimals" e "model.Menu.getNicks" sono invece usati da "view.SettingsPanel" per il caricamento di informazioni relative i nickname dei giocatori e i nomi degli avatar all'interno dei 2 JComboBox del pannello.

RIPRODUZIONE DELL'AUDIO

È stato fatto utilizzo della classe AudioManager fornita appositamente per la realizzazione del progetto. Tale classe è singleton e ha il compito di avviare l'audio tramite il metodo "play", che effettua una lettura bufferizzata del file passato in input. Affinché il metodo funzioni, è necessario che il nome del file passato come parametro sia contenuto nella cartella di gioco "resources" (cartella contenente anche le immagini usate per il gioco), e che l'estensione del file audio sia ".wav".

USO DI ANIMAZIONI ED EFFETTI SPECIALI

Le animazioni sono state usate per la rappresentazione del ResultPanel. Come già spiegato, questo pannello fa uso del pattern Decorator, e infatti oltre all'implementazione base del pannello, sono state create 3 classi concrete che definiscono 3 tipi diversi di decorazioni:

1. Mostrare il JLabel rappresentante il risultato carattere per carattere (effetto "typing")
2. Far sfocare gradualmente il testo del JLabel (effetto "fade")
3. Colorare semplicemente il testo

ALTRE NOTE PROGETTUALI E DI SVILUPPO

LINK DI ATTRIBUZIONE DELLE IMMAGINI AVATAR

- DOG: Dog icons created by Freepik - Flaticon
- GORILLA, PENGUIN, SNAKE, DEER, MONKEY, FOX, EAGLE, SHARK: Animal icons created by Freepik - Flaticon
- OWL: Owl icons created by Freepik - Flaticon
- RABBIT: Rabbit icons created by Freepik - Flaticon
- CAT: Cool icons created by BZZRINCANTATION - Flaticon
- WOLF: Wolf icons created by Freepik - Flaticon
- HORSE: Horse icons created by Freepik - Flaticon
- TIGER: Tiger icons created by Atif Arshad - Flaticon
- DEALER: Designed by Freepik

ATTRIBUZIONE DEL FILE AUDIO USATO

Audio tracks 1 to 9 are intellectual property of French Cafe 24x7 French Cafe Collection, vol. 6
2017 French Cafe 24x7 Released on: 2017-10-08:

- Track 1: Café de la Paix (starts at 00:00)
- Track 2: Chloé (starts at 4:05)
- Track 3: Beyond Seine (starts at 8:49)
- Track 4: Parisian summer (starts at 15:04)
- Track 5: Sweet as macaron (starts at 21:42)
- Track 6: Oh Françoise (starts at 25:30)
- Track 7: Some sunny day (starts at 29:21)
- Track 8: Bal au Moulin Rouge (starts at 35:23)
- Track 9: Velvet night (starts at 39:04)
- Track 10: Sweet dreams (starts at 43:48): French Cafe 24x7 French Cafe Collection, vol. 3
French Cafe 24x7 Released on: 2017-01-10 2017